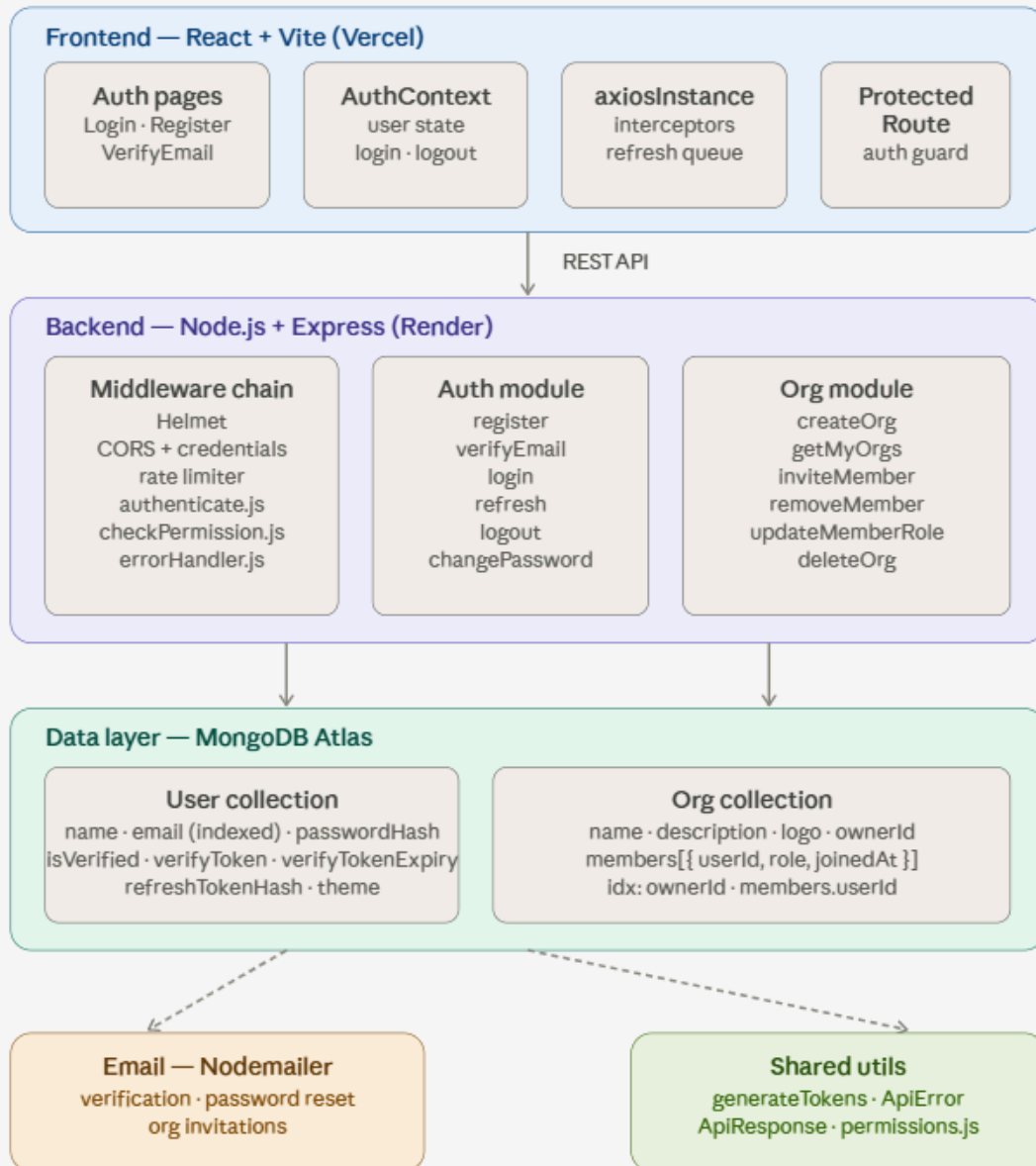


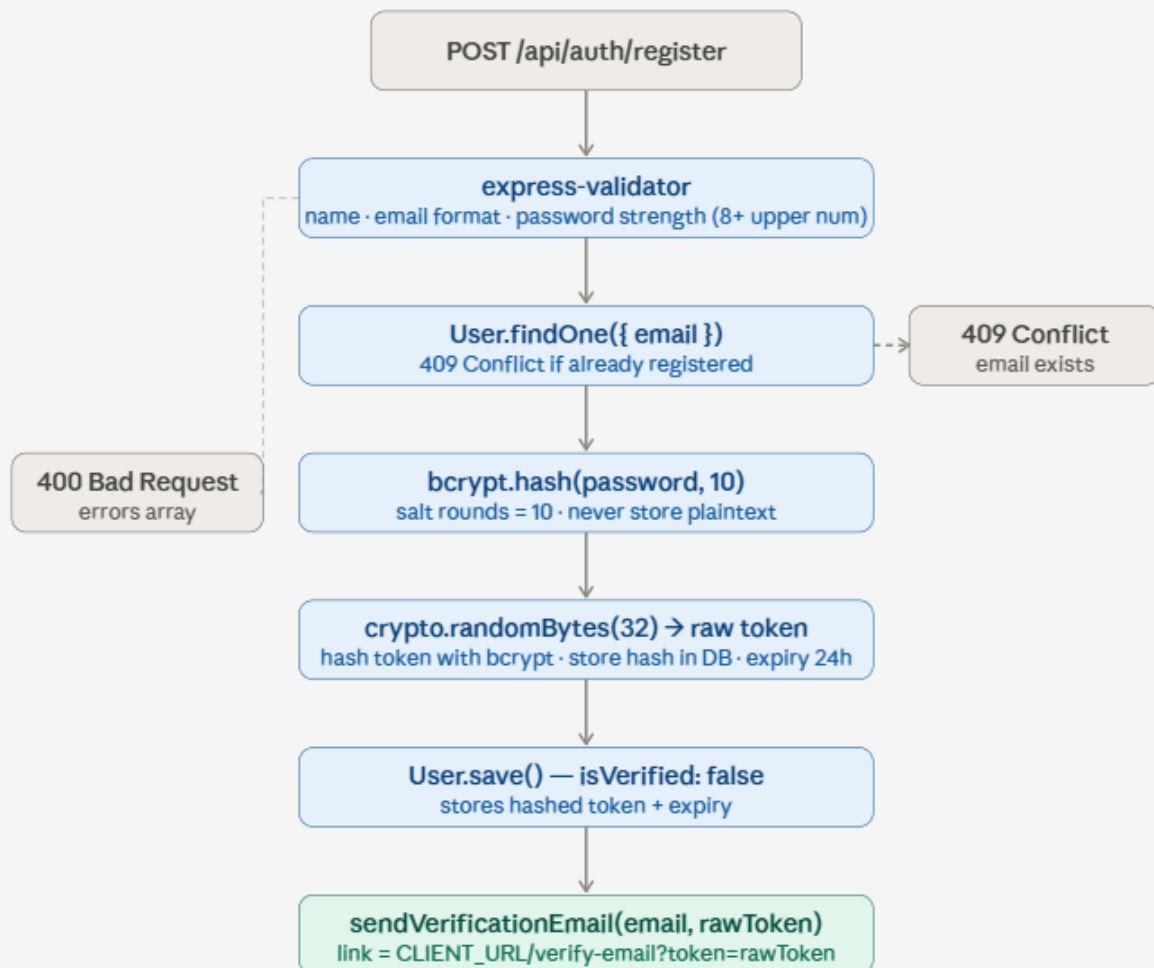
Phase 1 — complete system architecture

...



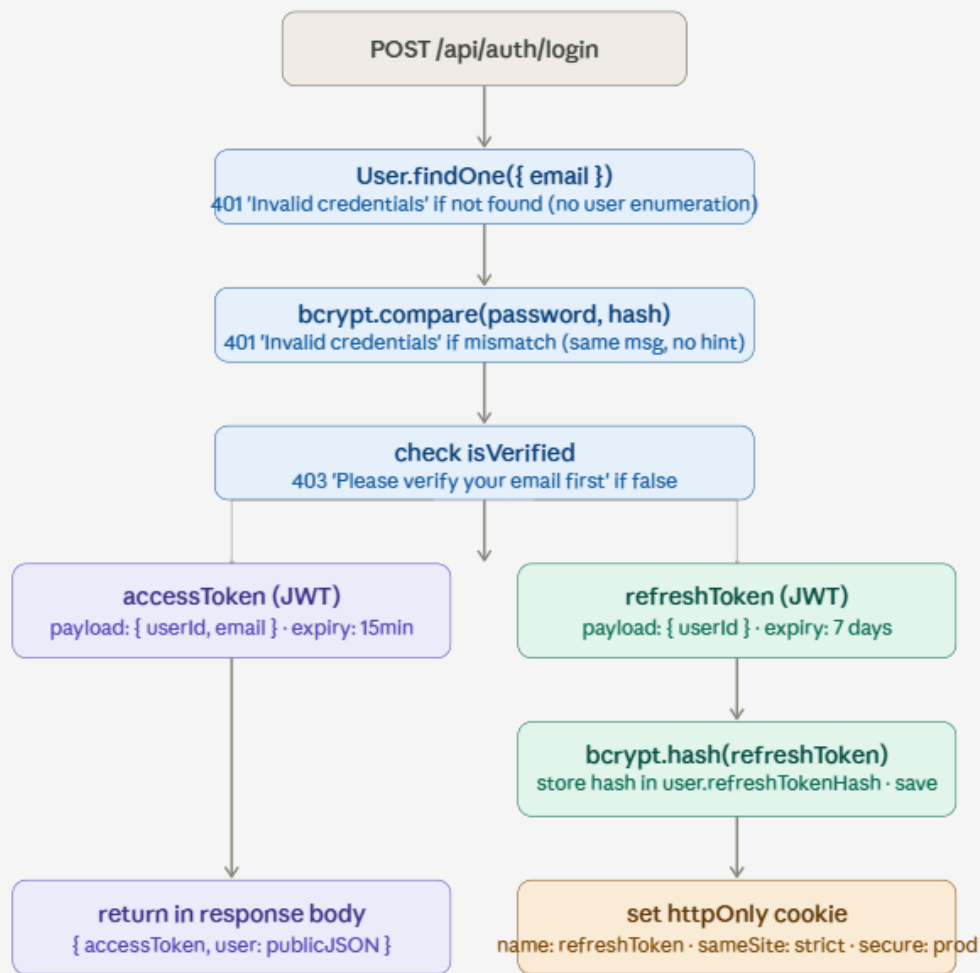
Now let's go through every flow an interviewer will ask about — with diagrams.

Flow 1 — Register + Email Verification



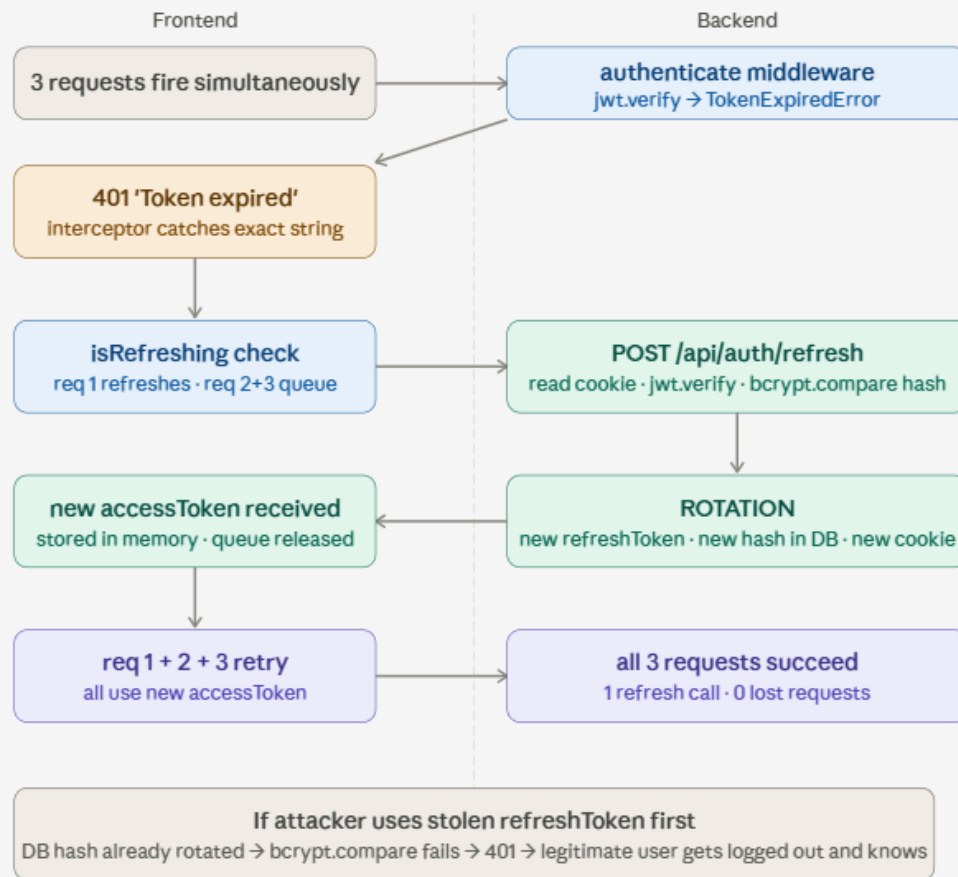
Interview explanation: "I store the verification token hashed in the DB — not the raw token. The raw token goes only in the email link. When the user clicks it, I `bcrypt.compare` the raw token from the URL against the hash in DB. This is the same principle as password hashing — even if someone reads your DB, they can't generate valid tokens."

Flow 2 — Login + JWT issuance



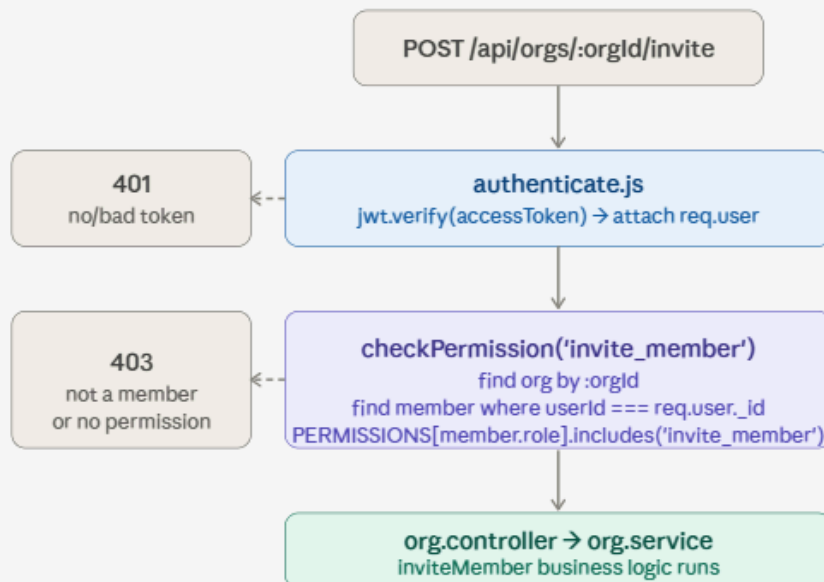
Interview explanation: "I return the same error message — 'Invalid credentials' — whether the email doesn't exist or the password is wrong. This is called preventing user enumeration. If I said 'email not found', an attacker could probe which emails are registered. The access token lives in memory on the frontend, never localStorage. The refresh token lives in an httpOnly cookie — JavaScript literally cannot read it, making XSS attacks unable to steal it."

Flow 3 — Token Refresh + Rotation



Interview explanation: "This solves two problems at once. The thunderbolt problem — multiple simultaneous requests with an expired token — is solved by the frontend queue. Only one refresh call goes out, all others wait. Token theft is solved by rotation — every refresh replaces the old refresh token. If an attacker steals and uses a refresh token, the next legitimate request finds the DB hash doesn't match and logs the user out. The window of risk is the 7-day cookie lifetime, narrowed to effectively zero by rotation."

Flow 4 — RBAC + checkPermission



Permission matrix (permissions.js)

owner	all actions including transfer_ownership
admin	all except transfer_ownership
manager	create_project · invite_member · manage_tasks · upload_files
developer	manage_tasks · upload_files
viewer	[] — read access only via route design

Interview explanation: "RBAC is implemented as a two-middleware chain. First `authenticate` verifies who you are. Then `checkPermission('action')` verifies what you're allowed to do. The permission check always looks up the role at the resource level — a user might be `owner` in one org and `viewer` in another. There's no global role. The permissions object is a single source of truth — adding a new role or action means editing one place, not hunting through controllers."

What Phase 1 contains — complete checklist

AUTH SYSTEM — WEEK 1

...

✓ **Register + email verification**
bcrypt hash · verify token hashed in DB · 24h expiry

✓ **Login + dual JWT issuance**
accessToken 15min in body · refreshToken 7d httpOnly cookie

✓ **Refresh token rotation**
every refresh replaces old token · old hash invalidated

✓ **Logout**
DB hash cleared · cookie cleared · token invalidated

✓ **User model**
all fields · toPublicJSON() · email index

✓ **authenticate middleware**
exact 'Token expired' string · attaches req.user

✓ **express-validator**
all auth routes validated · errors array returned

✓ **Email utility**
nodemailer · verify email · password reset · invite

ORG MANAGEMENT + RBAC — WEEK 2

✓ **Org model**
members embedded · ownerId · compound indexes

✓ **Permission matrix**
5 roles · all actions defined · single source of truth

✓ **checkPermission middleware**
resource-level check · orgId from params

✓ **Org CRUD**
create · get all mine · get by id · update · delete

✓ **Member management**
invite · remove · update role · owner-only delete

✓ **Security hardening**
Helmet · CORS credentials · rate limiter · auth limiter

FRONTEND — BOTH WEEKS

✓ **axiosInstance + interceptors**
refresh queue · thunderbolt protection · auto redirect

✓ **AuthContext + useAuth hook**
silent session restore on mount · login · logout

✓ **Auth pages**
Login · Register · VerifyEmail with react-hook-form

✓ **ProtectedRoute**
loading state · redirect if no user · Outlet

SHARED UTILITIES — USED EVERYWHERE

✓ **ApiError class**
statusCode · message · errors[] · success: false

✓ **ApiResponse class**
consistent shape · success auto from statusCode

✓ **errorHandler middleware**
global · 4-param · registered last in app.js

✓ **generateTokens utility**
access · refresh · setRefreshTokenCookie · clearCookie



The 8 questions interviewers will ask about Phase 1 — exact answers

"Why store the refresh token hash in DB and not the raw token?"

Because if you store the raw token and someone reads your DB (SQL injection, backup leak), they can use every refresh token directly. Storing the hash means even with full DB access, they can't reverse-engineer a valid token. Same principle as passwords. The raw token only ever exists in the httpOnly cookie and briefly in memory during rotation.

"What is token rotation and why does it matter?"

Every time a refresh token is used, it's immediately replaced with a new one. The old hash in DB is overwritten. If an attacker steals a refresh token and uses it before the legitimate user does, the legitimate user's next request gets a 401 — the hash doesn't match because the attacker already rotated it. This is the only way to detect refresh token theft.

"Why is the access token in memory and not localStorage?"

LocalStorage is accessible by any JavaScript on the page — including injected scripts via XSS. A memory variable in a JS module is not accessible outside that module's scope. So even if an attacker injects script, they can't reach the access token. The 15-minute expiry further limits the damage window.

"Why same error message for wrong email and wrong password?"

User enumeration protection. If I say "email not found", an attacker can probe every email in a dictionary and learn which ones have accounts. With "Invalid credentials" for both cases, they learn nothing about whether the email exists.

"How does RBAC work at the code level?"

`checkPermission('action')` is a middleware factory. It returns a middleware function that reads `orgId` from route params, finds the org, finds the calling user in the members array, reads their role, and checks `PERMISSIONS[role].includes(action)`. The permission matrix is a plain JavaScript object — a single source of truth. Adding a new role or a new action is a one-line edit.

"Why embed members inside the Org document instead of a separate collection?"

Member lists are always read with the org — there's no use case for reading members without knowing which org they belong to. Embedding avoids a join. The array won't grow unboundedly — a realistic org has dozens to hundreds of members, well within MongoDB's 16MB document limit. If it ever grew to thousands, I'd extract it — and I'd mention that tradeoff in the code comment.

"What happens when the access token expires mid-session?"

The frontend Axios response interceptor catches any 401 with the exact message "Token expired". The first such response triggers a refresh call. Any concurrent requests that also got 401 are queued as promises. When the refresh resolves, the queue processes — all queued requests retry with the new token. If the refresh itself fails (refresh token also expired or revoked), the interceptor clears auth state and redirects to `/login`.

"How does silent session restore work?"

On app mount, `AuthContext` calls `POST /auth/refresh` silently — not because the user did anything, but to check if they have a valid refresh token cookie from a previous session. If they do, we get back an access token and restore their session without showing a login screen. If the call fails with 401, we just leave `user` as null and show the login page. The user never sees a flash of the wrong screen because we check `isLoading` before rendering any route.

The data model — draw this on a whiteboard in 60 seconds

User

```
_id, name, email*, passwordHash, avatar  
isVerified, verifyToken†, verifyTokenExpiry  
refreshTokenHash†, theme  
* unique index    † hashed, not plaintext
```

Org

```
_id, name, description, logo  
ownerId → User  
members: [{ userId → User, role, joinedAt }]  
indexes: ownerId, members.userId
```

The relationship is: one User can be in many Orgs. One Org has many members, each with a role. The owner is also in the members array with role `'owner'` — this makes permission checks uniform — you always look in `members`, never separately at `ownerId`.

What's NOT in Phase 1 (and why that's fine to say)

If an interviewer asks about real-time, file upload, or AI — "those are in Phase 2 and 3. Phase 1 establishes the auth foundation that all other modules depend on. Every subsequent feature inherits the `authenticate` and `checkPermission` middleware I built here."

That answer shows architectural thinking — you built the security layer first, then features on top. That's exactly how production systems are built.